React useEffect Hook - YouTube Tutorial Script (80% English + Roman Urdu)

Intro

Assalam-o-Alaikum everyone!

Welcome back to the React JS complete course.

Aaj hum React ka ek bohat important aur interview favorite hook seekhne wale hain, aur us ka naam hai **useEffect Hook**.

Agar aap React developer banna chahte hain, ya APIs fetch karna, timers chalana, event listeners use karna, ya page load hone ke baad koi task perform karna chahte hain, to **useEffect** ko samajhna compulsory hai.

Is video ke end tak aap sirf syntax nahi, balke **real-world thinking** bhi seekh jayenge ke useEffect kab aur kyun use hota hai.

So without wasting any time, let's get started.

What is useEffect?

Sab se pehle question.

What is useEffect?

Simple definition.

> **useEffect allows us to perform side effects inside React components.**

Ab question ye hai.

Side Effect kya hota hai?

React ka main kaam hota hai UI render karna.

Lekin kuch kaam aise hote hain jo rendering ka part nahi hote.

For example:

- API call karna
- Database se data lana
- Timer start karna

- Browser title change karna
- Event Listener add karna
- Local Storage access karna

Ye sab React ke bahar wale kaam hain.

In sab ko hum kehte hain

Side Effects

Aur in side effects ko handle karne ke liye React hume **useEffect Hook** provide karta hai.

Real Life Analogy

Imagine karo.

Aap restaurant me waiter ho.

Customer order deta hai.

Kitchen food prepare karti hai.

Food ready hone ke baad waiter usay customer tak deliver karta hai.

React bhi exactly isi tarah kaam karta hai.

Step 1

React UI render karta hai.

Step 2

UI render complete hoti hai.

Step 3

Uske baad useEffect run hota hai.

Yani

First Render → Then Side Effects

Isi liye useEffect render ke baad execute hota hai.

Importing useEffect

Sab se pehle hook import karte hain.

```
import { useEffect } from "react";
```

React automatically nahi deta.

Isliye import karna zaroori hai.

Basic Syntax

```
useEffect(() => {

});
```

Yahan callback function pass hota hai.

Ye callback React render complete hone ke baad execute karta hai.

First Example

```
import { useEffect } from "react";

function App() {

    useEffect(() => {
        console.log("Component Rendered");
    });

    return (
        <h1>Hello React</h1>
    );

}

export default App;
```

Browser open karo.

Console dekho.

Output

```
Component Rendered
```

Ab agar component dobara render hoga.

Ye console phir execute hoga.

Why?

Question.

React ko kaise pata chalta hai ke callback kab chalana hai?

Answer.

React har render ke baad useEffect ko execute karta hai.

Default behavior yehi hota hai.

Different Behaviors of useEffect

React me useEffect teen major ways me use hota hai.

Hum ek ek karke samajhte hain.

Case 1

Run on Every Render

```
useEffect(() => {
    console.log("Runs on every render");
});
```

Dependency array nahi di.

Result.

Har render pe chalega.

Example.

State change hui.

Component render hua.

Effect bhi run hua.

Again state change.

Again render.

Again effect.

Case 2

Run Only Once

Ab maan lo hume sirf page load hone par API call karni hai.

Har render pe nahi.

To dependency array empty denge.

```
useEffect(() => {
    console.log("Runs Only Once");
}, []);
```

Empty dependency array ka matlab.

Sirf first render ke baad execute hoga.

Dobara nahi chalega.

Ye sabse common use case hai.

Mostly APIs isi tarah fetch hoti hain.

Case 3

Run When Dependency Changes

Ab maan lo humare paas counter hai.

```
const [count, setCount] = useState(0);
```

Aur hum chahte hain ke sirf count change hone par effect chale.

```
useEffect(() => {

    console.log("Count Changed");

}, [count]);
```

Ab kya hoga?

Count change hua.

Effect run.

Name change hua.

Effect nahi chalega.

Sirf count observe ho raha hai.

Isi ko dependency kehte hain.

Dependency Array

Dependency array React ko batata hai.

"Kis value ko observe karna hai."

```
[count]
```

React compare karega.

Old value.

New value.

Agar difference hua.

To callback execute hogi.

Agar difference nahi.

To callback skip ho jayegi.

Is wajah se unnecessary executions avoid hoti hain.

Performance improve hoti hai.

Complete Counter Example

```
import { useEffect, useState } from "react";

function App() {

    const [count, setCount] = useState(0);

    useEffect(() => {

        console.log("Count Changed");

    }, [count]);

    return (

        <div>

            <h1>{count}</h1>

            <button onClick={() => setCount(count + 1)}>
                Increase
            </button>

        </div>

    );

}

export default App;
```

Har button click pe.

Count update.

Component render.

Effect execute.

Simple.

Real World Example

Browser title change karna.

```
useEffect(() => {

    document.title = `Count ${count}`;

}, [count]);
```

Ab browser tab ka title automatically update hoga.

Ye real project me bohat common hai.

Fetch API Example

Sabse common example.

```
useEffect(() => {

    fetch("https://jsonplaceholder.typicode.com/users")
        .then(response => response.json())
        .then(data => console.log(data));

}, []);
```

Empty dependency array.

Matlab API sirf first render par call hogi.

Har render pe nahi.

Isi wajah se unnecessary requests nahi jayengi.

Cleanup Function

Ab ek important concept.

Cleanup Function.

Kab use hoti hai?

Jab component remove hone wala ho.

Ya dependency change hone wali ho.

Example.

```
useEffect(() => {

    console.log("Component Mounted");

    return () => {

        console.log("Component Unmounted");

    };

}, []);
```

Ye return function cleanup kehlata hai.

Ye automatically execute hota hai.

Mostly use hota hai.

- Timer clear karne ke liye
- Event Listener remove karne ke liye
- Memory leaks avoid karne ke liye
- WebSocket disconnect karne ke liye

Timer Example

```
useEffect(() => {

    const timer = setInterval(() => {

        console.log("Running");

    }, 1000);

    return () => {

        clearInterval(timer);

    };

}, []);
```

Agar cleanup nahi karoge.

Timer background me chalta rahega.

Aur application memory waste karegi.

Interview Question

Question.

Difference between

```
useEffect(() => {});
```

and

```
useEffect(() => {}, []);
```

Answer.

First one

Runs after every render.

Second one

Runs only after first render.

Common Mistakes

Mistake Number 1

Dependency array bhool jana.

Result.

Infinite re-renders ya unnecessary execution.

Mistake Number 2

State ko directly effect ke andar update karna without dependency.

Ye infinite loop create kar sakta hai.

Example.

```
useEffect(() => {
    setCount(count + 1);
});
```

Ye dangerous hai.

Har render ke baad state update hogi.

Phir render.

Phir update.

Infinite loop.

Mistake Number 3

Cleanup function use na karna.

Result.

Memory leaks.

Slow application.

Quick Revision

Yaad rakhne ka easiest formula.

No Dependency

↓

Every Render

Empty Dependency

↓

Only First Render

Specific Dependency

↓

Only When Value Changes

Return Function

↓

Cleanup

Outro

Congratulations!

Aaj aapne React ka ek sab se important hook master kar liya.

Ab aap samajh gaye hain:

- What is useEffect
- Side Effects
- Dependency Array
- Empty Dependency
- Specific Dependency
- API Fetching
- Cleanup Function
- Timer Example
- Browser Title Update
- Common Mistakes
- Interview Questions

Next video me hum practical projects ke through **multiple useEffect examples** implement karenge aur dekhenge ke real-world React applications me is hook ko professional way me kaise use kiya jata hai.

Agar video valuable lagi ho to Like karein, Channel Subscribe karein, aur bell icon zaroor press karein.

Thanks for watching.

Allah Hafiz.